Manufacturing Multimodal RAG Architecture

System design for a manufacturing maintenance assistant

1. System Overview

This project is designed as two connected layers: the ingestion layer and the query/AI layer. The ingestion layer processes manuals and prepares a searchable knowledge base. The AI layer reads that knowledge base and answers technician questions with citations and safety checks.

2. Ingestion Layer

The ingestion pipeline reads raw PDF manuals, extracts content, structures it by section, creates chunk records, generates embeddings, and stores processed results in cloud storage and BigQuery.

3. Visual Architecture

```
PDF files in data/raw/
    ?
main.py / pipeline.py
    ?
pdf_extractor.py
    ?? reads text from pages
    ?? finds tables
    ?? extracts images
    ?? keeps hyperlinks
    ?
relationship_builder.py
    ?
chunk_builder.py
    ?
embedding_generator.py
    ?
gcs_ingestor.py + bigquery_loader.py
    ?
processed files in data/processed/

TECHNICIAN / FRONTEND
    ?
API LAYER
    ?
QUERY AGENT SERVICE
    ?
request validation + machine context
    ?
query embedding
    ?
vector search in BigQuery
    ?
evidence ranking + deduplication
    ?
Gemini answer generation
    ?
structured response with citations + safety warnings
```

4. Main Components

? src/simple_ingestion/main.py - entry point for running the pipeline
? src/simple_ingestion/pipeline.py - coordinates one-file or batch processing
? src/simple_ingestion/pdf_extractor.py - extracts text, tables, images, and links
? src/simple_ingestion/table_extractor.py - handles merged cells and table normalization
? src/simple_ingestion/relationship_builder.py - groups content into document sections
? src/simple_ingestion/chunk_builder.py - creates searchable chunks
? src/simple_ingestion/embedding_generator.py - creates vector embeddings

? src/simple_ingestion/gcs_ingestor.py - uploads artifacts to GCS
? src/simple_ingestion/bigquery_loader.py - loads data into BigQuery
? src/simple_ingestion/config.py - environment and GCP settings

5. Query Flow

1. User sends a question with machine info, optional error code, and optional image.
2. The backend validates the request and contextualizes the machine.
3. A query embedding is generated.
4. Relevant chunks are retrieved using vector similarity and metadata filters.
5. The evidence is deduplicated and ranked.
6. A grounded answer is generated only when the retrieved evidence is sufficient.
7. The response includes citations, safety warnings, and a clarification question if needed.

6. Key Design Principles

? Separate ingestion from query serving.
? Reuse embeddings generated during ingestion.
? Keep retrieved chunk count limited and configurable.
? Fetch diagrams/images only when relevant.
? Prefer deterministic filtering before extra AI calls.
? Never invent unsupported machine values, torque limits, or repair procedures.

7. Output Structure

The system stores enriched manual data like:
? extracted text and tables
? section relationships
? chunked content for retrieval
? embeddings for search
? asset references for related images and diagrams
? BigQuery tables for storage and retrieval

8. Expected Result

The final result is a grounded manufacturing assistant that answers technician questions using real manual evidence, supports multimodal references when needed, and keeps costs low by avoiding repeated expensive processing at query time.